www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

28/3/17

UNIT-IV

BACK TRACKING

→ Many problems which deals with searching for a set of solutions ~~are~~ which ask for an optimal solution satisfying some constraints can be solved using back-tracking method.

→ Dynamic programming design method uses ~~root~~ drute force approach to find the optimal solution. Here drute force approach will use ~~always~~ all possible ways to find an optimal solution.

→ we can use backtracking method to find the optimal solution with less no.of steps. In this method, if we are trying to find the soln in one path and unable to get the solution then stop there, come back to previous step and choose alternate path to find the optimal solution.

→ many of the problems we solve using back-tracking method require that all the solutions satisfies

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

a complex set of constraints.

→ For any problem these constraints can be divided into two catogories.

1. Explicit constraint
2. Implicit constraint.

1. Explicit constraint:

→ Explicit constraints are the rules that restricts each 'x_i' to take on values only from a given set.

→ common examples of Explicit constraints are

$x_i \geq 0$, $x_i = 0$ or 1

2. Implicit constraint:

→ Implicit constraints are the rules that determine which of the tuples in the solution space of 'I' satisfies the criterian function.

*Applications of backtracking:

1. n-queens problem
2. Sum of Subsets
3. Graph colouring
4. Hamiltonean cycle.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

1. n-queens problems:

→ In this problem, n-queens are to be placed on an n×n chess board in such a way that no two queens are placed on same row, same column and same diagonal.

Ex: 4-queens problem
8-queens problem

*4-queens problem:

→ In this problem, 4-queens are to be placed on 4×4 chess board so that no two queens are placed on same row, column and diagonal.

Explicit constraint:

→ Let us consider the number the rows and columns of chess board as 1,2,3,4. Queens can also be numbered 1,2,3,4. So the values of set $S_i = \{1,2,3,4\}$. Therefore solution space contains '4^4' ways.

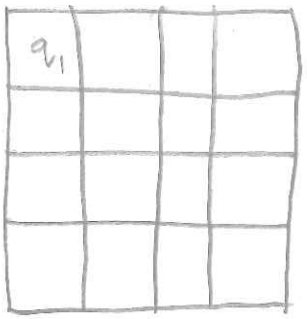
Implicit constraint:

→ No two queens are placed on same row, same column and same diagonal.

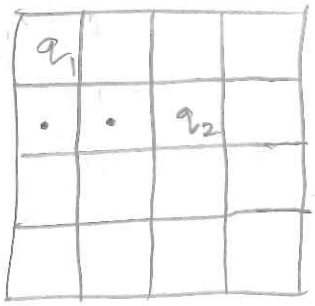
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

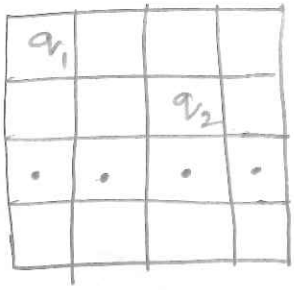
Step-1: Place queen 1 (q_1) on first row, first column.



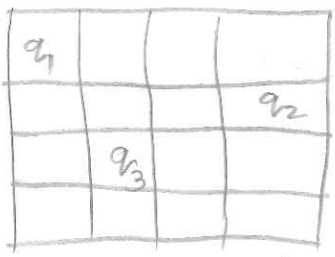
Step-2: Queen 2 (q_2) cannot be placed on second row first column and also second column. So place on third column.



Step-3: Queen 3(q_3) cannot be placed on any one of the columns



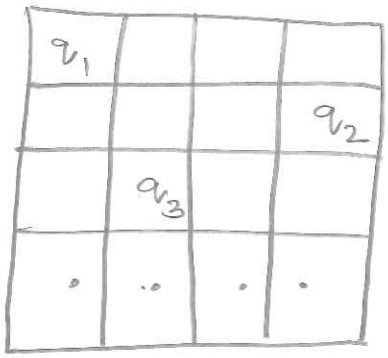
Backtrack to the previous step and place q_2 in another possible column i.e 4th column. and place q_3 in 2nd Column

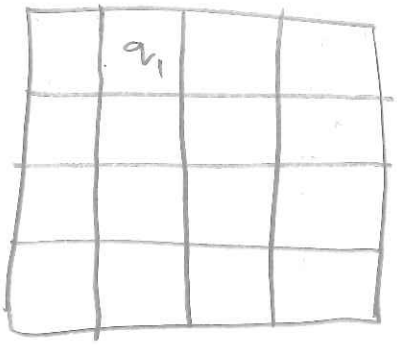


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

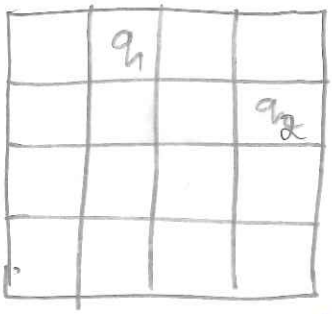
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-4: Queen 4 (q_4) cannot be placed on any of the columns. So backtrack to previous step q_3. All possible ways are completed for q_3 so backtrack to q_2. All possible ways are completed for q_2 also so backtrack to q_1. and change the position of q_1 from 1st column to second column.

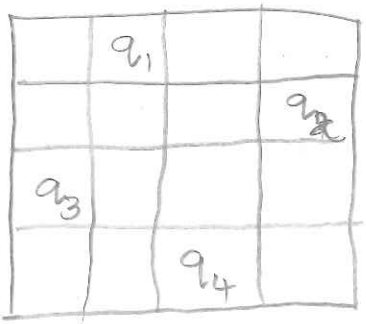




Step-5: Place q_2 on fourth column



Step-6: Place q_3 on 1st column & q_4 on 3rd column.



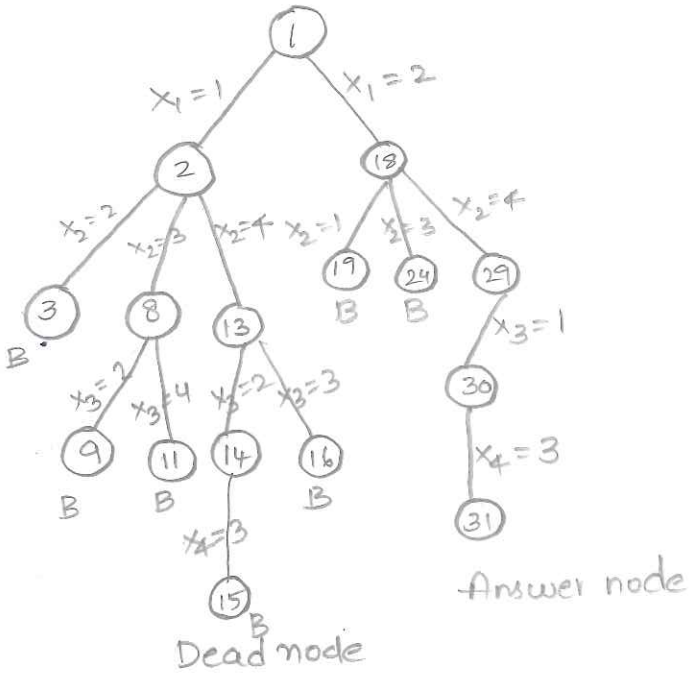
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Optimal solution:

$\{x_1, x_2, x_3, x_4\} = \{2, 4, 1, 3\}$

State space tree for 4-queens problem:



* Problem state:

Each node in state space tree defines a problem state

* State space:

All the paths from root node to other nodes define the state space of the problem.

* Solution state:

→ Solution state are problem states for which the path from the root to the other nodes defines a tuples in a solution space.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Answer State:

→ Answer states are the solution states for which the path from root to other nodes defines a tuples that is a member of the set of the solutions of the problem.

* State Space tree:

→ A tree organisation of solution space is represented as state space tree.

* Live node:

→ A node which has been generated and whose children have not yet been generated is called a live node

* E-node:

→ The live node whose children are currently been generated is called E-node.

* Dead node:

→ A dead node is a generated node which is not to be expanded further

* Bounding function:

→ These are used to kill live nodes without

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

generating all their children.

* 8-queen's problem:

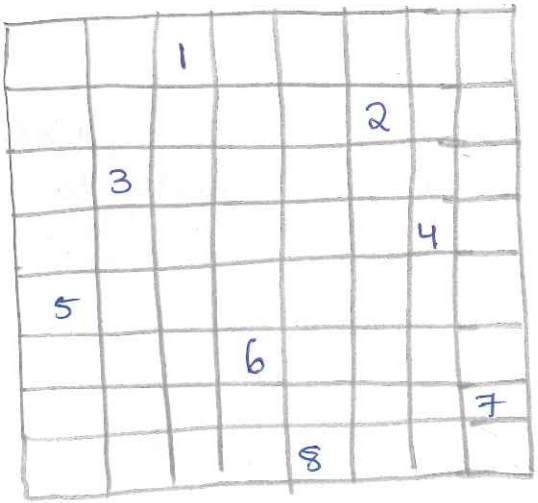
8-queen's prob is an objective function or criterian function. In this problem 8 queens are to be placed on 8x8 chess board so that no 2 queens placed on same row, same column and same diagonal.

Explicit constraint:

8x8 chess board contains 8 rows & 8 columns. The queens can also be numbered 1 to 8. So $S_i = \{1, 2, \ldots 8\}$ Therefore the solution space contains 8^8 tuples.

Implicit constraints:

No two queens are placed on same row, same column, same diagonal. One of solution's of 8 queen's problem.



$(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8) = (3, 6, 2, 7, 1, 4, 8, 5)$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Algorithm:

```
Algorithm place(k, i)
{
for j=1 to k-1 do
if ((x[j]= i) or (Abs(x[j]-i)= Abs(j-k)))
then return false;
return true
}
```

```
Algorithm NQueen(k, A)
{
for i:=1 to n do
{
if place(k, i) then
{
x[k]= i;
if (k==n) then write(x[1:n]);
else NQueens(k+1, n);
}
}
}
```

* Sum of subsets problem:

→ Suppose we are given n-distinct positive numbers and we need to find all combinations of this numbers whose sums are 'm'. This is called the sum of subsets problem.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Bounding function:

$$\sum_{i=1}^{k} w_i x_i + \sum_{i=k+1}^{n} w_i \geq m$$

Example: $n=4 \quad (w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$

$m = 31$

Find all combination of the number whose sum is m.

Sol:-

Subset	Sum	Remarks
{ }	0	
{11}	11	
{11,13}	11+13=24	
{11,13,24}	11+13+24=48	Exceeds m value backtrack
{11,13}	-	-
{11,13,7}	11+13+7=31	Optimal soln

→ The optimal solution

$(x_1, x_2, x_3, x_4) = (1, 1, 0, 1)$

$11+13+7 = 31$ [whose sum = m]

→ One more optimal solution is

$(x_1, x_2, x_3, x_4) = (0, 0, 1, 1)$

$24+7 = 31$ [whose sum = m]

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Recursive backtracking algorithm for SS problem:

```
Algorithm sumofsub(s, i, sum)
{
x[i]:= 1;
if (s+w[i]= m) then write(x[1:n])
else if (s+w[i]+w[i+1]≤m)
then sumofsub(s+w[i], i+1, sum-w[i]);
if((s+sum+w[i]≥m)and(s+w[i+1]≤m))then
{
x[i]:=0;
sumofsub(s, i+1, sum-w[i]);
}
}
```

www.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ This algorithm takes 3 parameters (s, i, sum

where s is values of subsets

i is position of values or elements

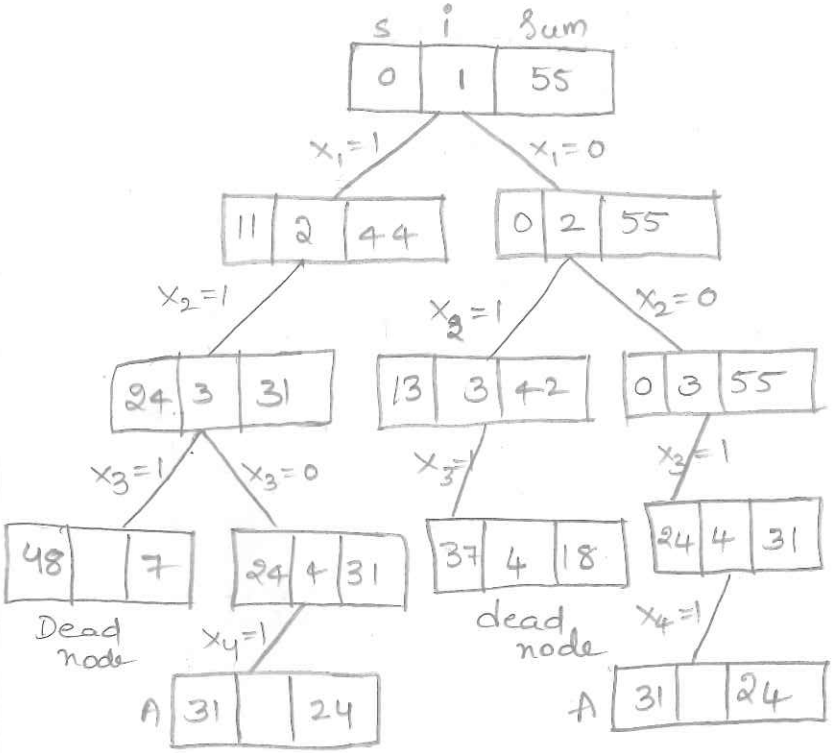
sum is sum of all elements of the given set.

for example: 1) $n = 4$

$(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$

$m = 31$

Draw the state space tree using recursive backtracking algorithm.



we got 2 optimal solutions subset (x_1, x_2, x_3, x_4)

$= (1, 1, 0, 1)$

$11 + 13 + 7 = 31$ [whose sum = m]

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

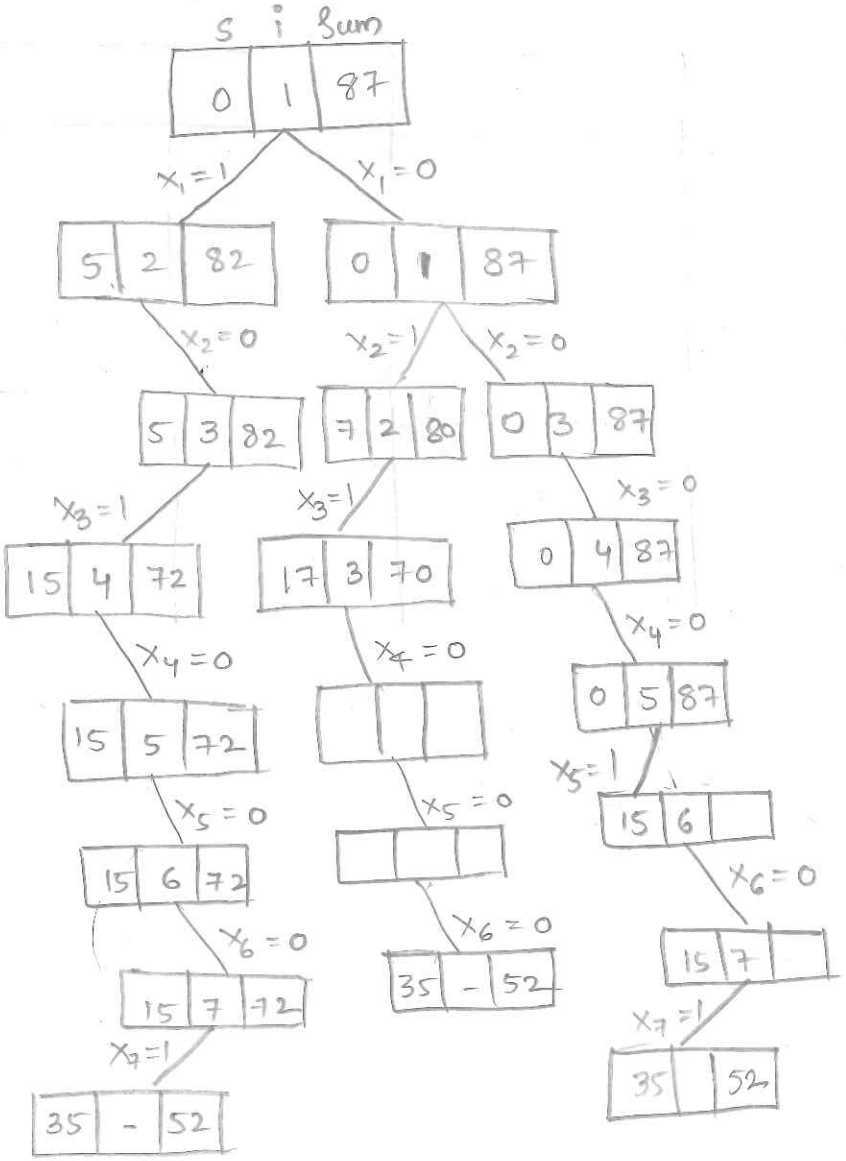
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Subset 2 = $(x_1, x_2, x_3, x_4) = (0, 0, 1, 1)$

$2_4 + 7 = 31$

2) Let $w = (5, 7, 10, 12, 15, 18, 20)$ and $m = 35$ find all possible subsets of w that sum is m. Draw the portion of state space tree

Sol:-



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* Graph coloring:

→ The process of coloring all vertices of a given graph in such a way that no two adjacent vertices have same colour yet only 'm' colours are used.

→ Note that if 'd' is the degree of the given graph, then it can be coloured with 'd+1' colours.

→ The m-colorability optimization problem asks for the smallest integer 'm' for which the graph 'G' can be coloured. This integer is referred to as the chromatic number of the graph.

→ A graph is said to be planar graph iff it can be drawn in a plane in such a way that no two edges cross each other.

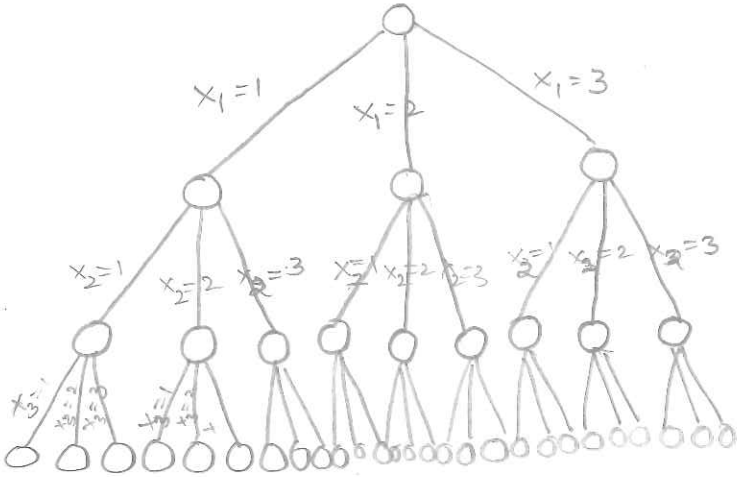
Ex:1) Draw a state space tree for m colouring. n=3,

m=3. n = no. of vertices
m = no. of colours.

Sol:- n=3
m=3

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



Algorithm:

finding all m-colorings of graph

```
Algorithm mcoloring(k)
{
 repeat
 {
 Nextvalue(k);
 if(x[k]=0) then return;
if (k=n) then
write (x[1:n]);
else mcoloring(k+1);
}until(false);
}
```

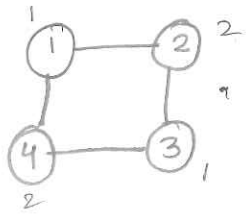
```
Algorithm Nextvalue(k)
{
 repeat
 {
  x[k]:= x[k]+1 mod(m+1);
if(x[k]=0) then return;
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

```
for j:=1 to n do
{
if((G[k,j]≠0) and (x[k]= x[j]))
then break;
}
if(j=n+1) then return
}until (false);
}
```

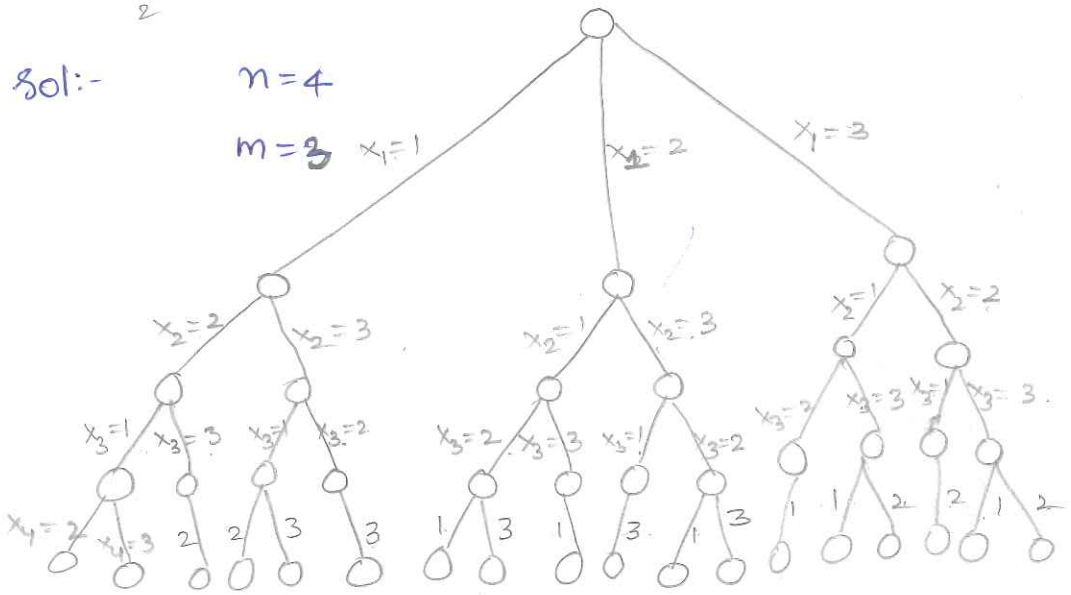
2) Color the given graph with m=3 colours



. Draw the state space tree.

Sol:- n=4

m=3



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

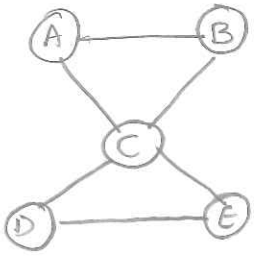
*Hamiltanion cycle:

→Let G = (V, E) be a connected graph with m-vertices.

*Hamiltonian path:

→ It is a path in a connected graph that visits each vertex exactly once.

Ex:



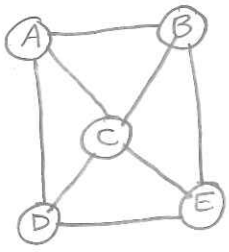
→ Hamiltanion path is A B C E D (or)
A B C D E.

→ Non-hamiltonion path is A B C E D C (or)
A B C D.

* Hamiltanion cycle:

→ Hamiltanion cycle is a cycle in a connected graph that visits each vertex exactly once and starting vertex, ending vertex must be same.

Ex:



→Hamiltanian cycle is A B C E D A (or)
A B E C D A

www.previousquestionpapers.com | https://telegram.me/jntuh

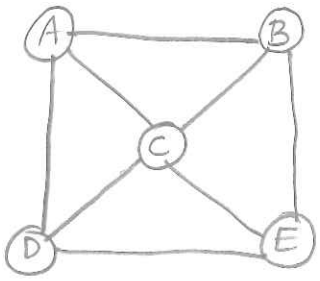
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→Not Hamiltanian cycle is ABCEBA

* Hamiltanian graph:

→Hamiltonian graph is a graph that contains hamiltonian cycle.

Ex:

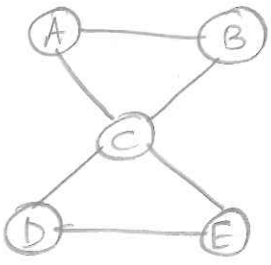


It consists of hamiltonian cycle. So, it is a hamiltonian graph.

*Traceable graph:

→Traceable graph is a graph that contains hamiltonian path is called traceable graph.

Ex:



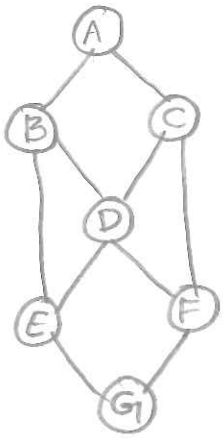
It consists of hamiltonian path ~~graph~~ so it is a traceable graph.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Problem-1:

Check whether this graph consists of hamiltonian cycle or not?

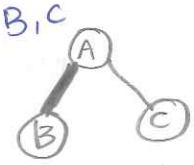


Sol:- Step-1: Visit the source vertex A.



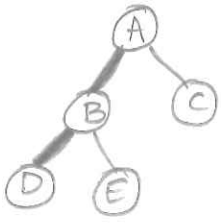
Step-2: Find all adjacent vertices of A and visit the nearest node.

A - B, C



Step-3: Find adjacent vertices of B and visit the nearest node. D

B - D, E.

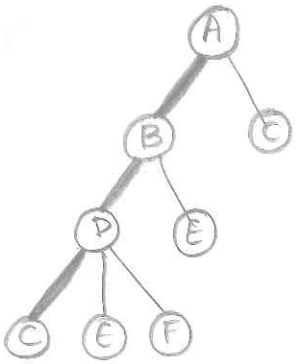


Step-4: Find all adjacent vertices of D (i.e) D - B, C, E, F but B is already visited so visit next nearest

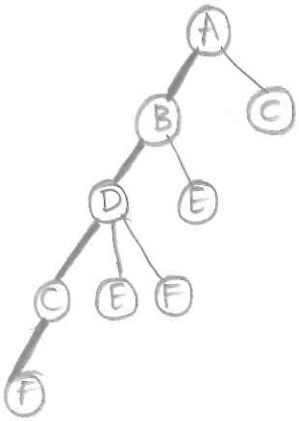
www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

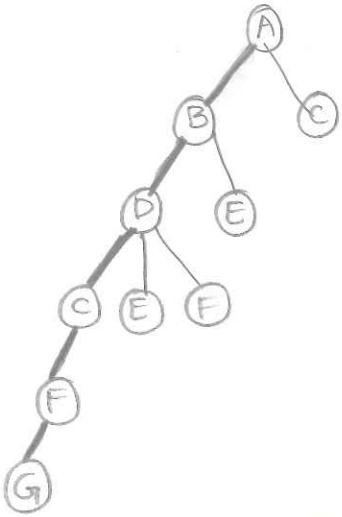
Vertex C.



Step-5: Find all adjacent vertices of C i.e C-A, D, F. Since A and D are already visited visit F.



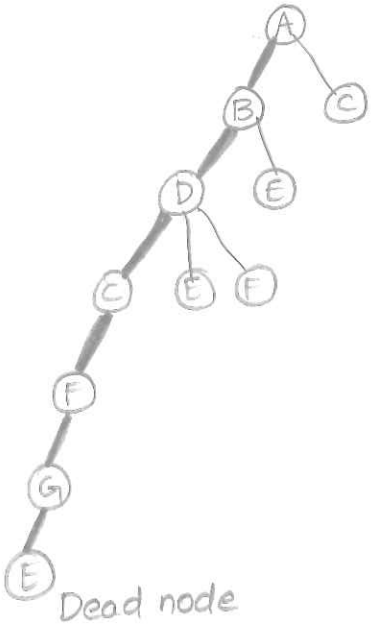
Step-6: Find all adjacent vertices of F i.e F-C, D, G. Since C and D are already visited visit 'G'.



Step-7: Find all adjacent vertices of G (i.e) G-E, F. F is already visited. visit E.

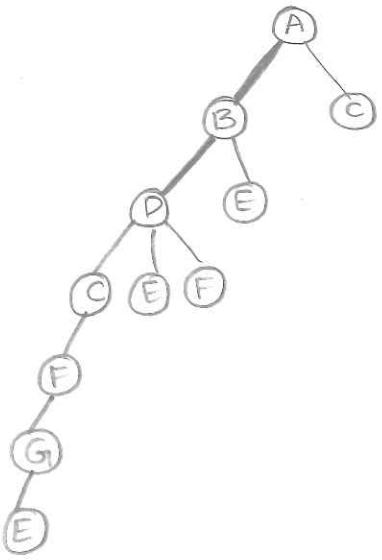
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



Dead node

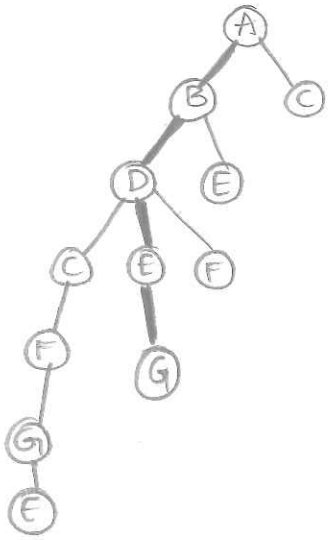
Step-8: Find all adjacent vertices of E i.e E - B, D, G. These vertices already visited and unable to generate child node. This node is known as dead node. Backtrack to vertex G there is no alternate path for G. So back track to F. no alternate path for vertex F also so backtrack to C. Since there is no alternate path for C backtrack to D. Now, choose alternate path and visit vertex E discard previous paths.



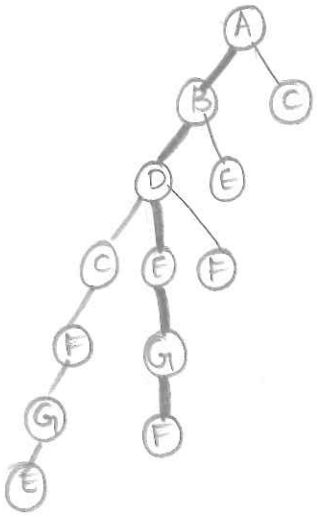
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

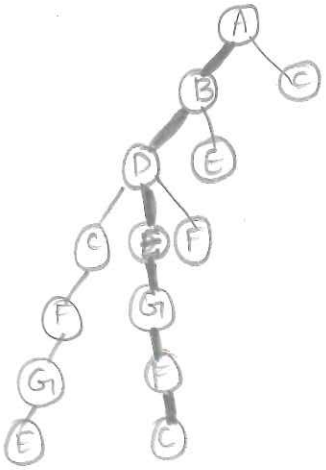
Step-9: Find all adjacent vertices of E (i.e) E - B, D, G visit G.



Step-10: Find all adjacent vertices of G (i.e) E, F. E is already visited so visit F.



Step-11: Find all adjacent vertices of F - G, C then visit C.

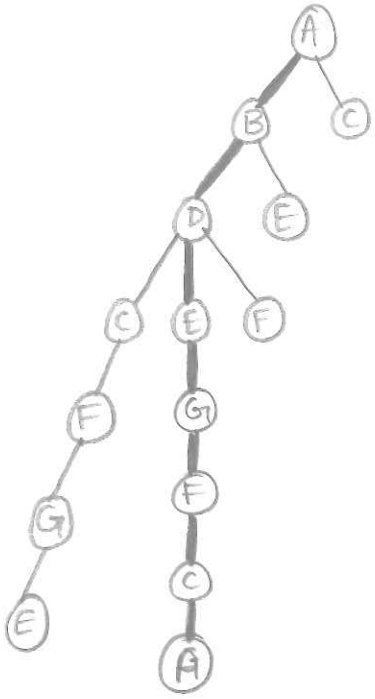


www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Step-12: Similarly find all adjacent vertices of C - A, D, F

Since D, F are already visited visit A.



Hamiltonian cycle: A B D E G F C A.

BRANCH AND BOUND

* Branch and bound:

→ The term branch and bound refers to all state space search methods in which all children of the E-node are generated before any other live node can become the E-node.

→ There are two graph search strategies.

1. BFS
2. DFS

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ In this methods, the expansion of a new node cannot begin until the node currently being the exploded is fully explored.

→ Both of this methods generalise to branch and bound strategies.

→ In Branch and bound terminology, BFS called as FIFO Search and DFS called as LIFO Search.

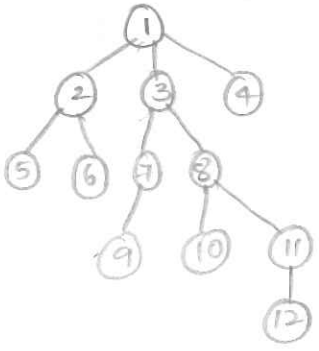
→ A branch and bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node.

→ There are three common search strategies.

1. First in first out (FIFO)
2. Last in first out (LIFO)
3. Least cost search (LC)

1. First in first out search (FIFO):

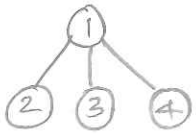
→ First in first out search method uses queues data structure. consider the following example.



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

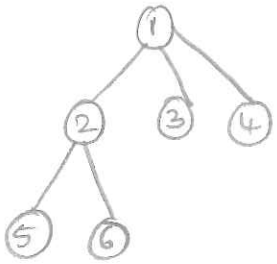
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Assume that there is only one node. This node becomes Enode. Expand and ~~extent~~ generate all its children 2,3,4 nodes. Insert this nodes into the queue.



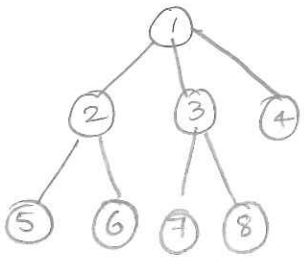
2	3	4

→ Queue follows FIFO. So remove node 2 from the queue and make it as E-node and generate its child nodes. 5,6 and insert 5,6 into queue



3	4	5	6

→ Make node 3 as E-node and generate it child nodes and insert into queue.



4	5	6	7	8

~~→ Make node 4 as E-node~~

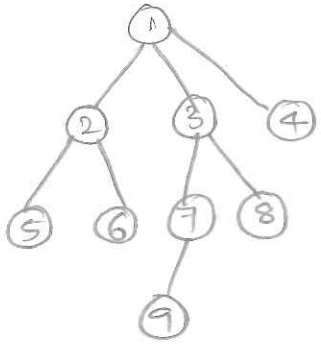
→ Nodes 4,5,6 killed by bounding function because they are not possible to generate child nodes. Now 4,5,6 are removed from queue. Now, queue contains 7,8

7	8

www.android.previousquestionpapers.com | www.universityupdates.in | https://telegram.me/jntuh

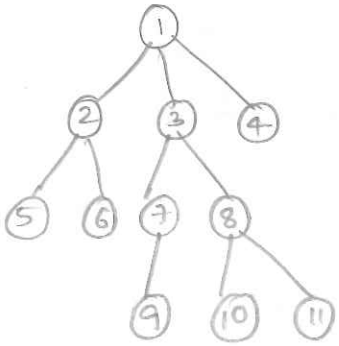
www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ Make 7 as E-node and generate its children and insert into queue.



8	9

→ Make 8 as E-node and generate its children and insert into queue

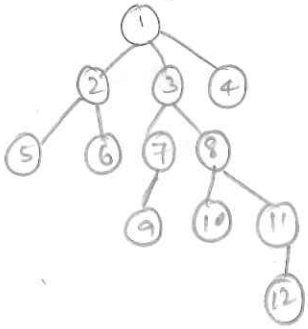


9	10	11

→ Make 9,10 as E-node since it does not generate children. i.e They are dead nodes. Remove 9,10 from queue.

11

→ Make 11 as the E-node and generate its child nodes and insert it into queue. 12 is the answer node and stop the process.



www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

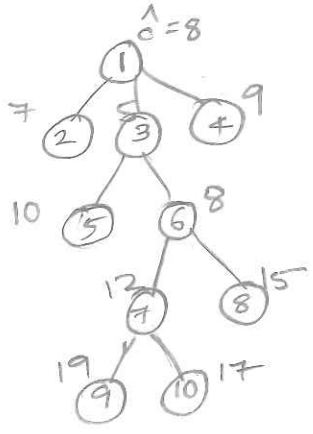
3. Least cost (LC):

→ In both FIFO and LIFO branch and bound the selection rule for next E-node is not efficient. The selection rule for next E-node does not give any preference to any node that has a very good chance of getting the search to an answer node quickly.

→ In least cost search, the next E-node is selected on the basis of ranking function ($\hat{c}()$).

→ Ranking function is used to find cost of each and every node. The node which has least cost is selected as E-node to generate child nodes. This process continues until the ~~defined~~ to find. answer node.

→ Consider the following example.



→ The cost of node 1 is 8 and make this node as E-node to generate child nodes 2, 3, 4. Find

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

the cost of this node using ranking function. The cost of nodes 2, 3, 4 are 7, 5, 9 respectively. The cost of node 3 is least and is selected as E-node to generate child nodes 5 and 6.

→ Find cost of 5 and 6 using ranking function. The node 6 cost is least(8) and select this node as E-node to generate child nodes 7 and 8.

→ Find cost of 7 and 8 using ranking function The costs are 12 and 15 respectively. The cost of node 7 is least and select this node as E-node to generate child nodes 9 & 10. The costs are 19 and 17 respectively. The cost of node 10 is least and it is answer node. Stop the process.

* Least cost algorithm:

→ Control abstraction for branch and bound using least cost search method:

```
Algorithm LCsearch (t)
{
if *t is an answer node then
    output *t and return;
  E := t, //E-node
```

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Initialize the list of live nodes to be empty

repeat

for each child x of E do

{

if x is an answer node then output the path

from x to t and return

Add(x);

(x → parent) := E;

}

if there are no more live nodes then write

("No answer node"); return;

E = least ( );

}until (false);

}

Algorithm

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

* 0/1 knapsack problem using least cost search method:

→ Branch and bound technique is used to solve minimization problems where as knapsack problem is a maximization problem. So convert maximization problem into minimization problem by replacing objective function $\Sigma P_i x_i$ by the function $-\Sigma P_i x_i$. The modified knapsack problem is stated as

$$\text{minimize} \; -\sum_{i=1}^{n} P_i x_i$$

$$\text{Subject to} \; \sum_{i=1}^{n} w_i x_i \leq m$$

$$x_i = 0 \text{ or } 1, \; 1 \leq i \leq n.$$

* LC branch and bound solution for knapsack problem:

1) Consider the knapsack instance $n=4$, $(P_1, P_2, P_3, P_4) = (10, 10, 12, 18)$, $(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$ and $m = 15$. Solve this problem using LC branch and bound.

Sol:- The search begins with root node as E-node

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Initially root node is node '1'. Find lower bound cost and upper bound cost for node 1 using ranking function.

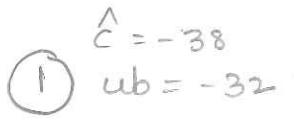
→ lower bound will accept the fractional values and upper bound does not accept fraction values.

Lower bound $(\hat{c}) = \langle P_i x_i \rangle = -(P_1x_1 + P_2x_2 + P_3x_3 + P_4x_4)$

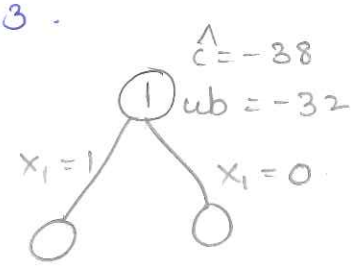
$\hat{c}(lb) = (10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9) = 38 = -38$

$ub = (10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0) = 32 = -32$

upper bound = -32.



→ Make node 1 as E-node to generate child nodes 2 and 3.



$x_1 = 1$ means first item must be placed in the knapsack.

$x_1 = 0$ means first item should not be placed in the knapsack.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ for node 2 $[x_1 = 1]$: First item must be placed in bag

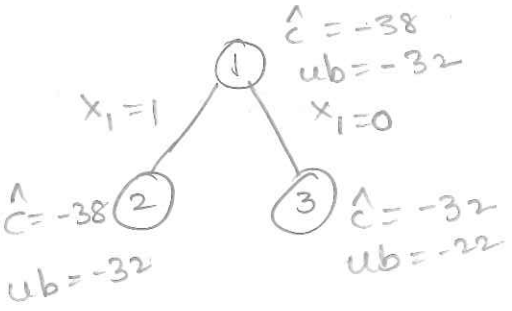
$\hat{c} = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9 = 38 = -38$

$ub = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = 32 = -32$

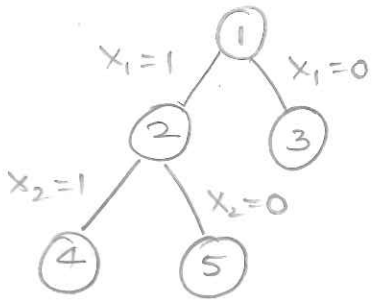
→ for node 3 $[x_1 = 0]$: first item should not be placed in bag.

$\hat{c} = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 5/9 = -32$

$ub = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = -22$



→ upper bound cost of node 2 is -32 which is least compared to node 3. So select node 2 as E-node to generate child nodes. 4 and 5



→ node 4 $[x_2 = 1]$: Second item must be placed in bag

$\hat{c} = lb = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9 = -38$

$ub = 10 \cdot 0 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = 32 = -32$

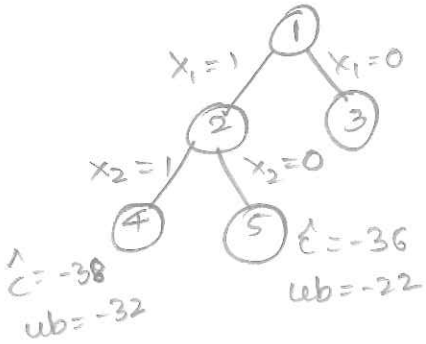
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

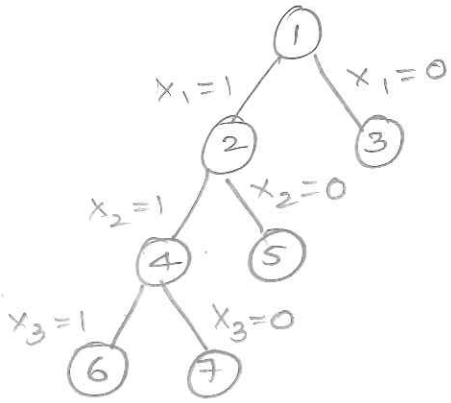
→ node 5 $[x_2 = 0]$: Second item should not be placed in bag

$$\hat{c} = lb = -(10 \cdot 1 + 10 \cdot 0 + 12 \cdot 1 + 18 \cdot 7/9) = -36$$

$$ub = -(10 \cdot 1 + 10 \cdot 0 + 12 \cdot 1 + 18 \cdot 0) = -22$$



→ upper bound cost of node 4 is -32 which is least compared to node 5. So select node 4 as E-node to generate its child nodes 6 & 7.



node 6 $[x_3 = 1]$:

$$\hat{c} = lb = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 3/9 = -38$$

$$ub = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 1 + 18 \cdot 0 = -32$$

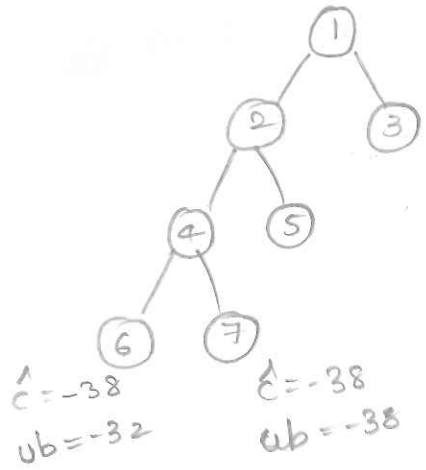
node 7 $[x_3 = 0]$:

$$\hat{c} = lb = -(10 \cdot 1 + 10 \cdot 1 + 18 \cdot 1) = -38$$

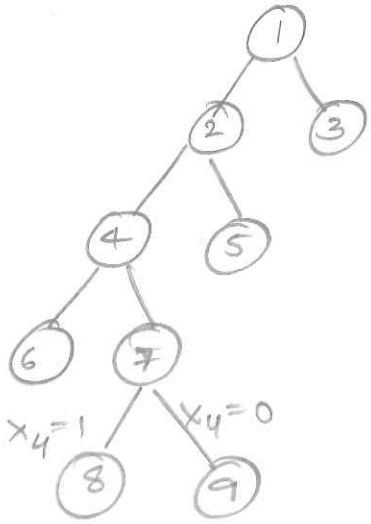
$$ub = -(10 \cdot 1 + 10 \cdot 1 + 18 \cdot 1) = -38$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



→ upper bound cost of node 7 is -38 which is least compared to node 6. So select node 7 as E-node to generate its child nodes 8 & 9



node 8 $[x_4 = 1]$:

$$\hat{c} = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 9/9 = -38$$

$$ub = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 9/9 = -38$$

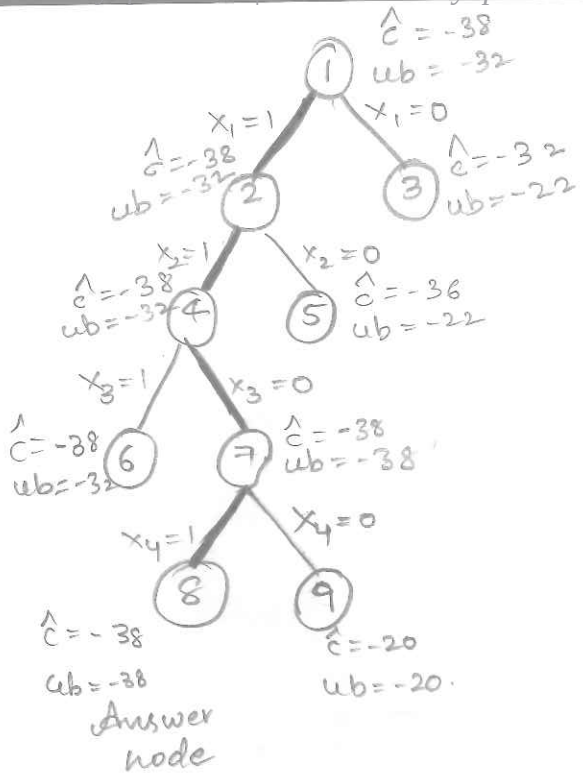
node 9 $[x_4 = 0]$:

$$\hat{c} = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 0 = -20$$

$$ub = 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 0 = -20$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh



→ubcost of node 8 is −38 which is least then compared to node 9 and select this node as answer node.

→ So optimal solution is $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 1$.

Maximum profit

$$\sum p_i x_i = P_1 x_1 + P_2 x_2 + P_3 x_3 + P_4 x_4$$

$$= 10 \cdot 1 + 10 \cdot 1 + 12 \cdot 0 + 18 \cdot 1$$

$$\sum p_i x_i = 38$$

*Travelling salesperson problem:

→Travelling salesperson problem consists of salesman and set of cities. A salesman has to visit every city, from starting a certain city(home city) and

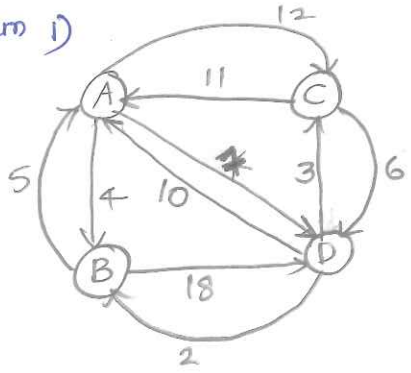
www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Come back to same city. The objective of this problem is that salesman wants to reduce cost of the trip.

→ Consider the following graph.

Problem 1)



Sol:- LC branch and bound solution.

Step-1: Represent the given graph in the form of adjacent cost matrix.

	A	B	C	D
A	∞	4	12	7
B	5	∞	∞	18
C	11	∞	∞	6
D	10	2	3	∞

Now reduce the cost matrix by using row reduction and column reduction.

Row reduction:

→ Subtract least element from all elements of the selected row

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

Initial

By row reduction

$$\begin{bmatrix} \infty & 4 & 12 & 7 \\ 5 & \infty & \infty & 18 \\ 11 & \infty & \infty & 6 \\ 10 & 2 & 3 & \infty \end{bmatrix} \begin{matrix} 4 \\ 5 \\ 6 \\ 2 \end{matrix} \quad 17 \Rightarrow \begin{bmatrix} \infty & 0 & 8 & 3 \\ 0 & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & 0 & 1 & \infty \end{bmatrix}$$

Column reduction:

→ Subtract least element from all elements of selected row

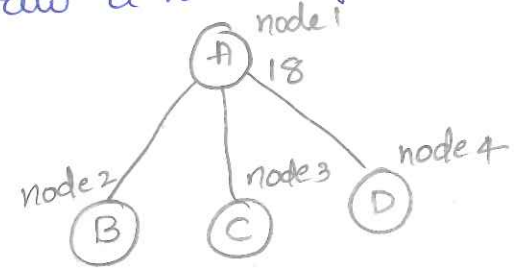
$$\begin{bmatrix} \infty & 0 & 8 & 3 \\ 0 & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & 0 & 1 & \infty \\ \hline 0 & 0 & 1 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} \infty & 0 & 7 & 3 \\ 0 & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & 0 & 0 & \infty \end{bmatrix}$$

→ So cost of node 1 is cost of row reduction + cost of column reduction

node1 cost = 17 + 1

= 18

→ Draw a state space tree



Step-2:

→ Choose vertex 'B' [node 2] to visit (path A → B)

1. From the reduced matrix of step-1 cost of [A, B] = 0

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

2. Set elements of row A and column B to infinite

3. Set cost of $[B, A] = \infty$.

4. The resulting cost matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & 0 & \infty \end{bmatrix}$$

Find the reduced cost matrix for above matrix.

Row reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & 0 & \infty \end{bmatrix} \begin{matrix} \\ \\ \\ 13 \\ 0 \\ 0 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & 0 & \infty \end{bmatrix}$$

13

Column reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ 5 & \infty & \infty & 0 \\ 8 & \infty & 0 & \infty \end{bmatrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ 0 & \infty & \infty & 0 \\ 3 & \infty & 0 & \infty \end{bmatrix}$$

5 0 0

reduced cost matrix

→ cost of node 2 = cost of node 1 + reduction cost + cost of $[A, B]$

$= 18 + (13 + 5) + 0 = 36.$

→ Now determine cost of node 3. choose vertex 3 (node 3) path $A \rightarrow C$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

1. From reduced matrix of step 1, cost $[A, C] = 7$

2. Set all elements of row A and column C to infinite

3. Set cost of $[C, A] = \infty$

4. Now resulting cost matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 13 \\ \infty & \infty & \infty & 0 \\ 8 & 0 & \infty & \infty \end{bmatrix}$$

→ This matrix is already reduced the cost of nodes row reduction and column reduction is already done in above matrix.

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 13 \\ \infty & \infty & \infty & 0 \\ 8 & 0 & \infty & \infty \end{bmatrix}$$

reduced matrix

cost of node 3 = cost of node 1 + reduction cost + cost $[A, C]$

$= 18 + 0 + 7 = 25.$

~~Step 3:~~

→ choose vertex D [node 4] path $[A \to D]$ 1. From the reduced matrix of step 'i' cost of $[A, D]$ is 3

2. Set all elements of row A and column D to ∞

3. Set cost $[D, A] = \infty$

4. ~~Cos~~ Resulting cost matrix is

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ 5 & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty \end{bmatrix}$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

row reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ 5 & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty \end{bmatrix} \begin{matrix} \infty \\ 0 \\ 5 \\ 0 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty \end{bmatrix}$$

This matrix is already column reduced.

Cost of node 4 = cost of node 1 + reduction cost + Cost of [A, D]

$= 18 + 5 + 3 = 26.$

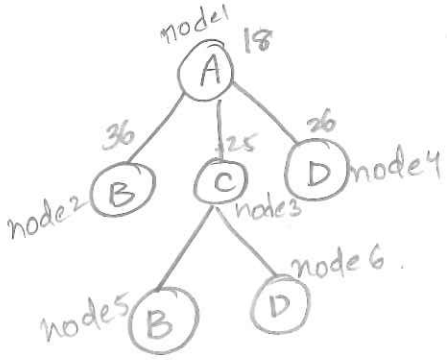
Cost of node 2 = 36 [A → B]

cost of node 3 = 25 [A → C]

cost of node 4 = 26 [A → D]

→ we have to choose the node with least cost

Therefore we choose node 3.



Step-3:

Note that we have to use cost matrix of node 3

(i.e) $\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 13 \\ \infty & \infty & \infty & 0 \\ 8 & 0 & \infty & \infty \end{bmatrix}$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

→ choose vertex B to visit from vertex C [A→C→B] is path.

1. From reduced matrix of step 2 cost of [C, B] = ∞
2. Set all elements of row C and column B to infinite.
3. Set cost of [B, A] to infinite.
4. Now resulting cost matrix is:

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ \infty & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty \end{bmatrix}$$

row reduction

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 13 \\ \infty & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty \end{bmatrix} \begin{matrix} \bullet \\ 13 \\ \\ 8 \\ 21 \end{matrix} \Rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix}$$

This matrix is already column reduced.

cost of node 5 = cost of node 3 + reduction cost + cost(C, B)

$$= 25 + 21 + \infty$$

$$= \infty.$$

→ choose vertex D (node 6) to visit (A → C → D)

1. From reduced matrix of step 2 cost of [C, D] = 0
2. Set all elements of row C and column D to infinite
3. Set cost of [D, A] = ∞
4. The resulting cost matrix is

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty \end{bmatrix}$$

This matrix is already reduced so that

cost of node 6 = cost of node 3 + reduction cost + cost of [C, D]

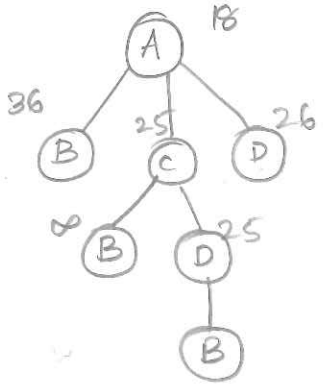
= 25 + 0 + 0

= 25.

→ Thus we have cost(5) = ∞, cost(6) = 25

path [A → C → D]

→ Now Lc node is 6 select this node as E-node to generate child nodes.



→ choose vertex B [node 7] path [A → C → D → B]

1. From reduced matrix of step-3 cost of [D, B] = 0
2. Set all elements of row D and column B to infinite
3. Set cost of [B, A] = ∞
4. The resultant matrix is

www.android.previousquestionpapers.com | www.previousquestionpapers.com | https://telegram.me/jntuh

www.android.universityupdates.in | www.universityupdates.in | https://telegram.me/jntuh

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \end{bmatrix}$$

→ This matrix is already reduced. So

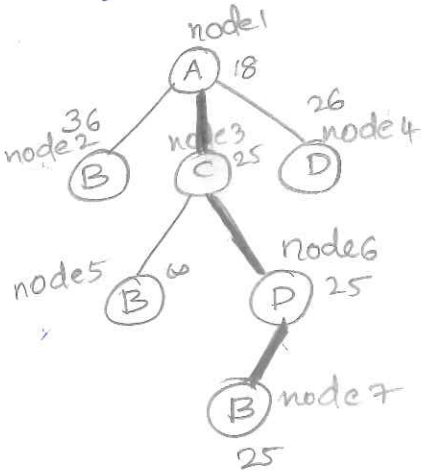
cost of node 7 = cost of node 6 + reduction cost + cost [D, B]

= 25 + 0 + 0

= 25

∴ cost of node 7 = 25

→ Thus optimal path is A → C → D → B → A with the cost of 25.



Solve travelling salesperson problem using FIFO.

www.android.previousquestionpapers.com | www.previous... /telegram.me/jntuh